

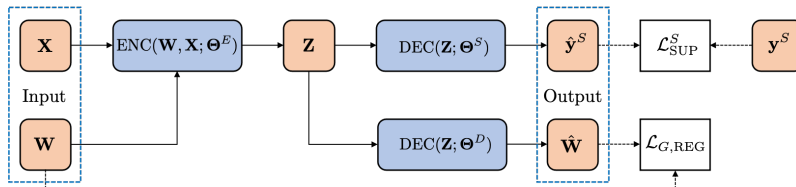
Graph Convolutional Neural Networks

Elvin Isufi

Intelligent Systems Department

May 15, 2026

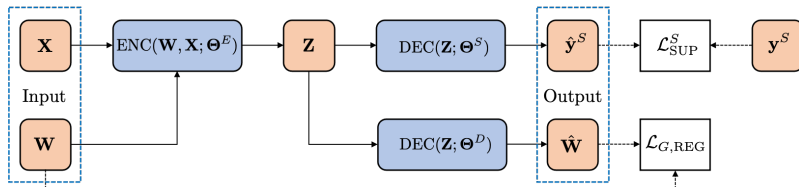
A Graph Machine Learning Pipeline



► GML methods require:

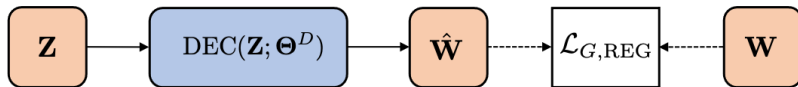
- ⇒ **Inputs:** a graph structure $\mathbf{W}$ and (potentially) features $\mathbf{X}$
- ⇒ **A graph-based function:** $\text{ENC}(\cdot)$ and $\text{DEC}(\cdot)$
- ⇒ **Loss function:** to measure the success of the task

A Graph Machine Learning Pipeline



- GML methods require:
 - ⇒ **Inputs:** a graph structure $\mathbf{W}$ and (potentially) features $\mathbf{X}$
 - ⇒ **A graph-based function:** $\text{ENC}(\cdot)$ and $\text{DEC}(\cdot)$
 - ⇒ **Loss function:** to measure the success of the task
- Building a GML method requires specifying how functions $\text{ENC}(\cdot)$ and $\text{DEC}(\cdot)$ capture the data-structure coupling

A Graph Machine Learning Pipeline



► Graph neural networks:

- ⇒ Learn hierarchical representation
- ⇒ Make both encoder and decoder parametric
- ⇒ Leverage both the signals/features and the topology
- ⇒ Inductive and scalable

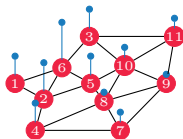
Outline: What will you learn today?

- ▶ What are graph convolutional neural networks?
 - ⇒ How the convolution operator extends to the graph setting?
 - ⇒ How to incorporate multiple features?

Graph Convolutional Filters

Graph Signal Shifting

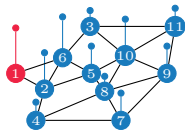
- ▶ **Graph** $\mathcal{G} = (\mathcal{V}, \mathcal{E}) \Rightarrow \mathcal{V}$: set of nodes, $\mathcal{E}$: set of edges
- ▶ **Graph signals** $\Rightarrow$ Associate a **value** to each node $\mathbf{x} : \mathcal{V} \rightarrow \mathbb{R}$
- ▶ **Matrix representation**: Graph shift operator $\mathbf{S} \Rightarrow$ Adjacency matrix $\mathbf{A}$, Laplacian matrix $\mathbf{L}$



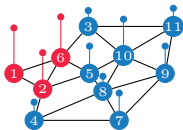
$$\mathbf{L} = \begin{bmatrix} 2 & -1 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ -1 & 4 & 0 & -1 & -1 & -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 3 & 0 & 0 & -1 & 0 & 0 & 0 & -1 & -1 \\ 0 & -1 & 0 & 3 & 0 & 0 & -1 & -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & 4 & -1 & 0 & -1 & 0 & -1 & 0 \\ -1 & -1 & -1 & 0 & -1 & 4 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & 0 & 0 & 3 & -1 & -1 & 0 & 0 \\ 0 & 0 & 0 & -1 & -1 & 0 & -1 & 5 & -1 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & -1 & -1 & 4 & -1 & -1 \\ 0 & 0 & -1 & 0 & -1 & 0 & 0 & -1 & -1 & 5 & -1 \\ 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & -1 & -1 & 3 \end{bmatrix}$$

Graph Signal Shifting Sequence

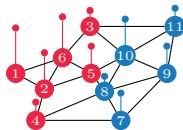
- Given a signal $\mathbf{x} = [1 \ 0 \ 0 \ \dots \ 0]^\top$



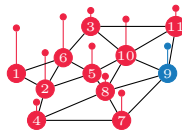
$\mathbf{x}$



$\mathbf{S}\mathbf{x}$



$\mathbf{S}^2\mathbf{x}$

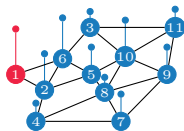


$\mathbf{S}^3\mathbf{x}$

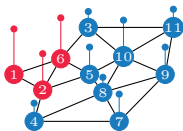
- K successive shifts imply exchanging information from neighbors up to K -hops away

Graph Signal Shifting Sequence

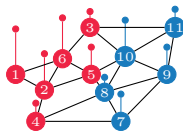
- Given a signal $\mathbf{x} = [1 \ 0 \ 0 \ \dots \ 0]^\top$



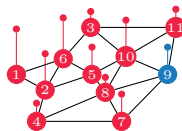
$\mathbf{x}$



$\mathbf{Sx}$



$\mathbf{S}^2\mathbf{x}$

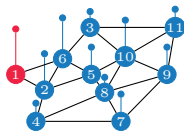


$\mathbf{S}^3\mathbf{x}$

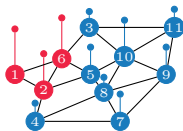
- K successive shifts imply exchanging information from neighbors up to K -hops away
 $\Rightarrow$ one shift $\mathbf{x}^{(1)} := \mathbf{Sx}$

Graph Signal Shifting Sequence

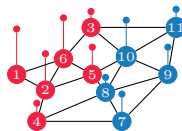
- Given a signal $\mathbf{x} = [1 \ 0 \ 0 \ \dots \ 0]^\top$



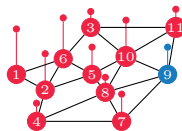
$\mathbf{x}$



$\mathbf{S}\mathbf{x}$



$\mathbf{S}^2\mathbf{x}$

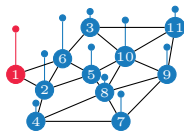


$\mathbf{S}^3\mathbf{x}$

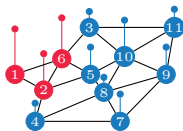
- K successive shifts imply exchanging information from neighbors up to K -hops away
 - $\Rightarrow$ one shift $\mathbf{x}^{(1)} := \mathbf{S}\mathbf{x}$
 - $\Rightarrow$ two shifts $\mathbf{x}^{(2)} := \mathbf{S}^2\mathbf{x} = \mathbf{S}(\mathbf{S}\mathbf{x}) = \mathbf{S}\mathbf{x}^{(1)}$

Graph Signal Shifting Sequence

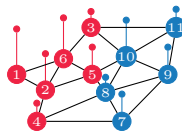
- Given a signal $\mathbf{x} = [1 \ 0 \ 0 \ \dots \ 0]^\top$



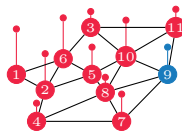
$\mathbf{x}$



$\mathbf{S}\mathbf{x}$



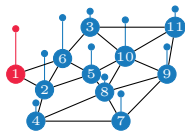
$\mathbf{S}^2\mathbf{x}$



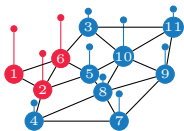
$\mathbf{S}^3\mathbf{x}$

- K successive shifts imply exchanging information from neighbors up to K -hops away
- ⇒ one shift $\mathbf{x}^{(1)} := \mathbf{S}\mathbf{x}$
 - ⇒ two shifts $\mathbf{x}^{(2)} := \mathbf{S}^2\mathbf{x} = \mathbf{S}(\mathbf{S}\mathbf{x}) = \mathbf{S}\mathbf{x}^{(1)}$
 - ⇒ K shifts $\mathbf{x}^{(K)} := \mathbf{S}^K\mathbf{x} = \mathbf{S}(\mathbf{S}^{K-1}\mathbf{x}) = \mathbf{S}\mathbf{x}^{(K-1)}$

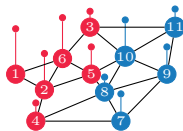
Graph Convolutional Filter



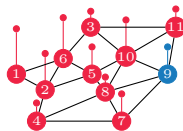
x



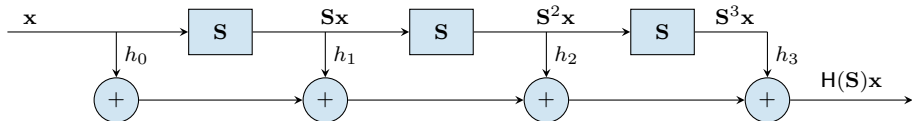
Sx



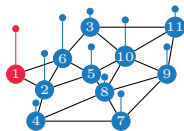
S^2x



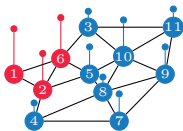
S^3x



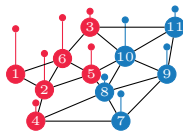
Graph Convolutional Filter



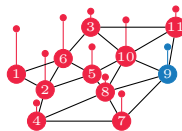
$\mathbf{x}$



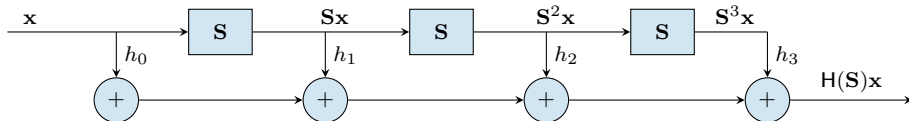
$\mathbf{Sx}$



$\mathbf{S}^2\mathbf{x}$



$\mathbf{S}^3\mathbf{x}$

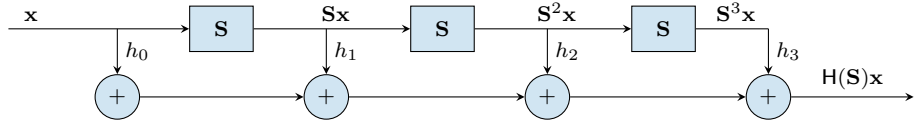
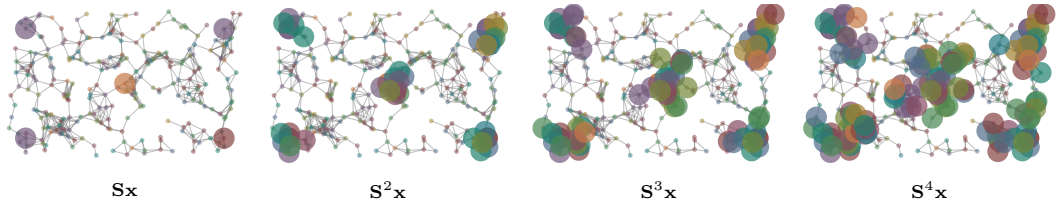


► **Graph convolution** $\Rightarrow$ **Linear combination** of shifted versions of the signal

$$\mathbf{H}(\mathbf{S})\mathbf{x} = \sum_{k=0}^K h_k \mathbf{S}^k \mathbf{x}$$

$\Rightarrow$ **Linear combination of neighboring signal** $\Rightarrow$ Local operation

Graph Convolutional Filter (cont.)



► Graph convolutional filter: $H(S) = \sum_{k=0}^K h_k S^k$

Properties of the Graph Convolutional Filters

- ▶ Graph convolutional filter: $\mathbf{z} = \sum_{k=0}^K h_k \mathbf{S}^k \mathbf{x} = h_0 \mathbf{x} + h_1 \mathbf{S} \mathbf{x} + h_2 \mathbf{S}^2 \mathbf{x} + \dots + h_K \mathbf{S}^K \mathbf{x}$

Properties of the Graph Convolutional Filters

- ▶ Graph convolutional filter: $\mathbf{z} = \sum_{k=0}^K h_k \mathbf{S}^k \mathbf{x} = h_0 \mathbf{x} + h_1 \mathbf{S} \mathbf{x} + h_2 \mathbf{S}^2 \mathbf{x} + \dots + h_K \mathbf{S}^K \mathbf{x}$
- ▶ Filter parameters: $h_0, \dots, h_K$
 - $\Rightarrow$ Each indicates the importance of a neighborhood of hop k
 - $\Rightarrow K$ is a hyperparameter and independent from the graph dimension

Properties of the Graph Convolutional Filters

- ▶ Graph convolutional filter: $\mathbf{z} = \sum_{k=0}^K h_k \mathbf{S}^k \mathbf{x} = h_0 \mathbf{x} + h_1 \mathbf{S} \mathbf{x} + h_2 \mathbf{S}^2 \mathbf{x} + \dots + h_K \mathbf{S}^K \mathbf{x}$
- ▶ Filter parameters: $h_0, \dots, h_K$
 - ⇒ Each indicates the importance of a neighborhood of hop k
 - ⇒ K is a hyperparameter and independent from the graph dimension
 - ⇒ Sharable parameters over the nodes
 - ⇒ Learn h_k implies learn a graph convolution

Properties of the Graph Convolutional Filters

- ▶ **Graph convolutional filter:** $\mathbf{z} = \sum_{k=0}^K h_k \mathbf{S}^k \mathbf{x} = h_0 \mathbf{x} + h_1 \mathbf{S} \mathbf{x} + h_2 \mathbf{S}^2 \mathbf{x} + \dots + h_K \mathbf{S}^K \mathbf{x}$
- ▶ **Filter parameters:** $h_0, \dots, h_K$
 - ⇒ Each indicates the importance of a neighborhood of hop k
 - ⇒ K is a hyperparameter and independent from the graph dimension
 - ⇒ **Sharable** parameters over the nodes
 - ⇒ **Learn h_k** implies **learn a graph convolution**
- ▶ **Q:** Can a **graph convolutional filter** be trained on a **graph $\mathcal{G}_1$** and deployed on another **graph $\mathcal{G}_2$** ? (**inductive property**)

Properties of the Graph Convolutional Filters

- ▶ **Graph convolutional filter:** $\mathbf{z} = \sum_{k=0}^K h_k \mathbf{S}^k \mathbf{x} = h_0 \mathbf{x} + h_1 \mathbf{S} \mathbf{x} + h_2 \mathbf{S}^2 \mathbf{x} + \dots + h_K \mathbf{S}^K \mathbf{x}$
- ▶ **Filter parameters:** $h_0, \dots, h_K$
 - ⇒ Each indicates the importance of a neighborhood of hop k
 - ⇒ K is a hyperparameter and independent from the graph dimension
 - ⇒ **Sharable** parameters over the nodes
 - ⇒ **Learn h_k** implies **learn a graph convolution**
- ▶ **Q:** Can a **graph convolutional filter** be trained on a **graph $\mathcal{G}_1$** and deployed on another **graph $\mathcal{G}_2$** ? (**inductive property**)
- ▶ **Computing the output: $\mathbf{y}$**
 - ⇒ Requires **matrix-matrix** multiplication $\mathbf{S}^k$
 - ⇒ complexity $\mathcal{O}(N^3)$ – impractical even for $N \sim 1000$

Properties of the Graph Convolutional Filters

- ▶ Graph convolutional filter: $\mathbf{z} = \sum_{k=0}^K h_k \mathbf{S}^k \mathbf{x} = h_0 \mathbf{x} + h_1 \mathbf{S} \mathbf{x} + h_2 \mathbf{S}^2 \mathbf{x} + \dots + h_K \mathbf{S}^K \mathbf{x}$
- ▶ Filter parameters: $h_0, \dots, h_K$
 - ⇒ Each indicates the importance of a neighborhood of hop k
 - ⇒ K is a hyperparameter and independent from the graph dimension
 - ⇒ Sharable parameters over the nodes
 - ⇒ Learn h_k implies learn a graph convolution
- ▶ Q: Can a graph convolutional filter be trained on a graph $\mathcal{G}_1$ and deployed on another graph $\mathcal{G}_2$? (inductive property)
- ▶ Computing the output: $\mathbf{y}$
 - ⇒ Requires matrix-matrix multiplication $\mathbf{S}^k$
 - ⇒ complexity $\mathcal{O}(N^3)$ – impractical even for $N \sim 1000$
 - ⇒ But we do not need $\mathbf{S}^k \Rightarrow$ we need $\mathbf{S}^k \mathbf{x}$
 - ⇒ Exploit the recursive property $\mathbf{x}^{(k)} = \mathbf{S}^k \mathbf{x} = \mathbf{S}(\mathbf{S}^{k-1} \mathbf{x}) = \mathbf{S} \mathbf{x}^{(k-1)}$

Learning with Graph Convolutional Filters (GCFs)

- ▶ You can already solve GML tasks with GCFs
- ▶ We are given:
 - ⇒ A training set $\mathcal{T}$ containing observations $\mathbf{x} \in \mathcal{T}$. can be supervised etc
 - ⇒ A loss function $\mathcal{L}(\hat{\mathbf{y}})$ to evaluate how good the estimate $\hat{\mathbf{y}}$ is can contain regularizers etc
 - ⇒ A function class $\mathcal{C}$ which could be that of the filters

Learning with Graph Convolutional Filters (GCFs)

- ▶ You can already solve GML tasks with GCFs
- ▶ We are given:
 - ⇒ A training set $\mathcal{T}$ containing observations $\mathbf{x} \in \mathcal{T}$. can be supervised etc
 - ⇒ A loss function $\mathcal{L}(\hat{\mathbf{y}})$ to evaluate how good the estimate $\hat{\mathbf{y}}$ is can contain regularizers etc
 - ⇒ A function class $\mathcal{C}$ which could be that of the filters
- ▶ Learning means finding function $\Phi^* \in \mathcal{C}$ that minimizes loss $\mathcal{L}(\Phi(\mathbf{x}))$ averaged over training set

$$\Phi^* = \arg \min_{\Phi \in \mathcal{C}} \sum_{\mathbf{x} \in \mathcal{T}} \mathcal{L}(\Phi(\mathbf{x}))$$

Learning with Graph Convolutional Filters (GCFs)

- ▶ You can already solve GML tasks with GCFs
- ▶ We are given:
 - ⇒ A training set $\mathcal{T}$ containing observations $\mathbf{x} \in \mathcal{T}$. can be supervised etc
 - ⇒ A loss function $\mathcal{L}(\hat{\mathbf{y}})$ to evaluate how good the estimate $\hat{\mathbf{y}}$ is can contain regularizers etc
 - ⇒ A function class $\mathcal{C}$ which could be that of the filters
- ▶ Learning means finding function $\Phi^* \in \mathcal{C}$ that minimizes loss $\mathcal{L}(\Phi(\mathbf{x}))$ averaged over training set

$$\Phi^* = \arg \min_{\Phi \in \mathcal{C}} \sum_{\mathbf{x} \in \mathcal{T}} \mathcal{L}(\Phi(\mathbf{x}))$$

- ▶ We use $\Phi^*(\mathbf{x})$ to estimate outputs $\hat{\mathbf{y}} = \Phi^*(\mathbf{x})$ on inputs $\mathbf{x}$ not observed in the training set $\mathcal{T}$

Learning with GCFs (cont.)

- The **function class** $\mathcal{C}$ is the degree of freedom available to the system's designer

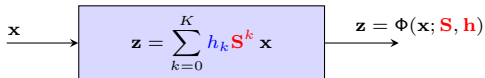
$$\Phi^* = \arg \min_{\Phi \in \mathcal{C}} \sum_{\mathbf{x} \in \mathcal{T}} \mathcal{L}(\Phi(\mathbf{x}))$$

Learning with GCFs (cont.)

- The **function class $\mathcal{C}$** is the degree of freedom available to the system's designer

$$\Phi^* = \arg \min_{\Phi \in \mathcal{C}} \sum_{\mathbf{x} \in \mathcal{T}} \mathcal{L}(\Phi(\mathbf{x}))$$

- Choose function class as **graph filters of order K** supported on **$\mathbf{S}$** : $\Phi(\mathbf{x}) = \Phi(\mathbf{x}; \mathbf{S}, \mathbf{h}) = \sum_{k=0}^K h_k \mathbf{S}^k \mathbf{x}$

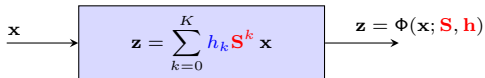


Learning with GCFs (cont.)

- The **function class $\mathcal{C}$** is the degree of freedom available to the system's designer

$$\Phi^* = \arg \min_{\Phi \in \mathcal{C}} \sum_{\mathbf{x} \in \mathcal{T}} \mathcal{L}(\Phi(\mathbf{x}))$$

- Choose function class as **graph filters of order K** supported on **$\mathbf{S}$** : $\Phi(\mathbf{x}) = \Phi(\mathbf{x}; \mathbf{S}, \mathbf{h}) = \sum_{k=0}^K h_k \mathbf{S}^k \mathbf{x}$



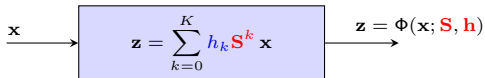
- **Learning solution restricted to graph filter class** $\Rightarrow \mathbf{h}^* = \arg \min_{\mathbf{h}} \sum_{\mathbf{x} \in \mathcal{T}} \mathcal{L}(\Phi(\mathbf{x}; \mathbf{S}, \mathbf{h}))$
 $\Rightarrow$ **Optimization is over filter coefficients $\mathbf{h}$** with the **graph shift operator $\mathbf{S}$** given

Learning with GCFs (cont.)

- ▶ The **function class $\mathcal{C}$** is the degree of freedom available to the system's designer

$$\Phi^* = \arg \min_{\Phi \in \mathcal{C}} \sum_{\mathbf{x} \in \mathcal{T}} \mathcal{L}(\Phi(\mathbf{x}))$$

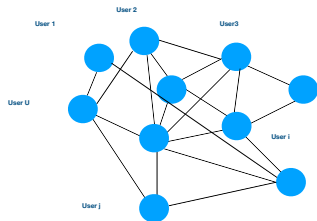
- ▶ Choose function class as **graph filters of order K** supported on **$\mathbf{S}$** : $\Phi(\mathbf{x}) = \Phi(\mathbf{x}; \mathbf{S}, \mathbf{h}) = \sum_{k=0}^K h_k \mathbf{S}^k \mathbf{x}$



- ▶ **Learning solution restricted to graph filter class** $\Rightarrow \mathbf{h}^* = \arg \min_{\mathbf{h}} \sum_{\mathbf{x} \in \mathcal{T}} \mathcal{L}(\Phi(\mathbf{x}; \mathbf{S}, \mathbf{h}))$
 $\Rightarrow$ **Optimization is over filter coefficients $\mathbf{h}$** with the **graph shift operator $\mathbf{S}$** given
- ▶ **Q:** What is $\text{ENC}(\cdot)$ and $\text{DEC}(\cdot)$ here?

Example: Graph Collaborative Filtering (cont.)

	Pulp Fiction	Avatar	Titanic	...	Friends
User 1	5	1	?	...	2
User 2	?	?	3	...	3
User 3	4	?	4	...	?
...	...	...	...	...	...
User U	?	4	3	...	1



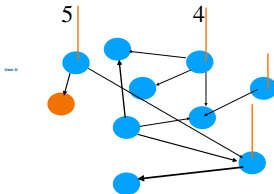
- Construct an **user-user graph** from the ratings
 - ⇒ connect two similar users based on Pearson correlation or other criteria/metadata

Example: Graph Collaborative Filtering (cont.)

- ▶ Item-specific user graph $\mathcal{G}_i = \{\mathcal{U}, \mathcal{E}_i\}$
 - $\Rightarrow$ item-specific shift operator $\mathbf{S}_i$
 - $\Rightarrow$ item-specific graph signal $\mathbf{x}_i$
- ▶ User-user nearest neighbour predicts ratings as

$$\mathbf{z}_i = \mathbf{S}_i \mathbf{x}_i$$

$\Rightarrow$ rating propagates from neighbours to missing values accounting for the strength



Example: Graph Collaborative Filtering (cont.)

- User-user nearest neighbour predicts ratings as

$$\mathbf{z}_i = \mathbf{S}_i \mathbf{x}_i$$

⇒ order one convolutional filter

⇒ $h_0 = 0, h_1 = 1$ - fixed parameters/not learned

Example: Graph Collaborative Filtering (cont.)

- ▶ User-user nearest neighbour predicts ratings as

$$\mathbf{z}_i = \mathbf{S}_i \mathbf{x}_i$$

⇒ order one convolutional filter

⇒ $h_0 = 0, h_1 = 1$ - fixed parameters/not learned

- ▶ Incorporate multiresolution information via order K convolutions

$$\mathbf{z}_i = \sum_{k=0}^K h_k \mathbf{S}_i^k \mathbf{x}_i$$

⇒ learn multi-hop parameters $h_0, \dots, h_K$

Example: Graph Collaborative Filtering

	Pulp Fiction	Avatar	Titanic	...	Friends
User 1	5	1	?	...	2
User 2	?	?	3	...	3
User 3	4	?	4	...	?
...	...	...	...	...	...
User U	?	4	3	...	1

- Training set $\mathcal{T}$: ratings X_{ui} from matrix $\mathbf{X}$

$$\underset{\mathbf{h}}{\text{minimize}} \quad \sum_{(u,i) \in \mathcal{T}} \left(\left[\sum_{k=0}^K h_k \mathbf{S}_i^k \mathbf{x}_i \right]_u - X_{ui} \right)^2 + \gamma \|\mathbf{h}\|_2^2$$

Example: Graph Collaborative Filtering

	Pulp Fiction	Avatar	Titanic	...	Friends
User 1	5	1	?	...	2
User 2	?	?	3	...	3
User 3	4	?	4	...	?
...	...	...	...	...	...
User U	?	4	3	...	1

- Training set $\mathcal{T}$: ratings X_{ui} from matrix $\mathbf{X}$

$$\underset{\mathbf{h}}{\text{minimize}} \quad \sum_{(u,i) \in \mathcal{T}} \left(\left[\sum_{k=0}^K h_k \mathbf{S}_i^k \mathbf{x}_i \right]_u - X_{ui} \right)^2 + \gamma \|\mathbf{h}\|_2^2$$

⇒ fitting term: $\sum_{(u,i) \in \mathcal{T}} \left(\left[\sum_{k=0}^K h_k \mathbf{S}_i^k \mathbf{x}_i \right]_u - X_{ui} \right)^2$

⇒ regularizer: $\gamma \|\mathbf{h}\|_2^2$ avoids overfitting

⇒ find filter coefficients $\mathbf{h}$

⇒ convex problem – least squares - closed form solution

Learning activity: Recommender systems with graph filters

Q2.1: A movie platform builds a **user-user graph** by connecting users with similar rating behavior. For one movie, the ratings form a graph signal, and missing ratings are predicted using

$$\mathbf{z}_i = \sum_{k=0}^K h_k \mathbf{S}_i^k \mathbf{x}_i.$$

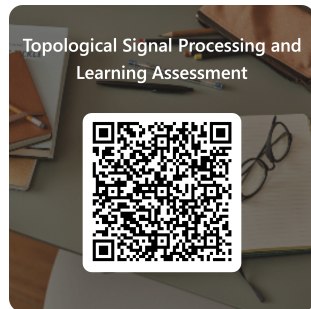
What is the role of the coefficients $h_0, \dots, h_K$?

► Instructions

- ⇒ Scan the QR code to open the activity
- ⇒ Work individually first
- ⇒ Then discuss your answer in pairs or groups of 3
- ⇒ Submit your final answer in Microsoft Forms

► Logistics

- ⇒ **Time:** 10 min
- ⇒ **Goal:** Understand how graph filters use multi-hop neighborhood information
- ⇒ **Product:** One submitted response per student



<https://forms.office.com/e/yTag1Njapt>

Graph Convolutional Neural Networks

Graph Convolutional Neural Networks

- ▶ Graph filters are great but:

- ⇒ are linear functions

- ⇒ parameters may be limited

- ⇒ increasing the order may not lead to better results Why?

Graph Convolutional Neural Networks

► Graph filters are great but:

- ⇒ are linear functions
- ⇒ parameters may be limited
- ⇒ increasing the order may not lead to better results Why?
- ⇒ they learn only **low-level** features (e.g., transform signal into some output)
- ⇒ useful for signal denoising and interpolation but **not** necessarily for node classification or more non-linear tasks.

Graph Convolutional Neural Networks

- ▶ Graph filters are great but:

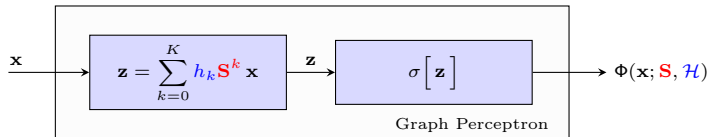
- ⇒ are linear functions
- ⇒ parameters may be limited
- ⇒ increasing the order may not lead to better results Why?
- ⇒ they learn only low-level features (e.g., transform signal into some output)
- ⇒ useful for signal denoising and interpolation but not necessarily for node classification or more non-linear tasks.

- ▶ Idea:

- ⇒ Use GCFs to build graph convolutional neural networks in a way CNNs are built for images

Graph Perceptron

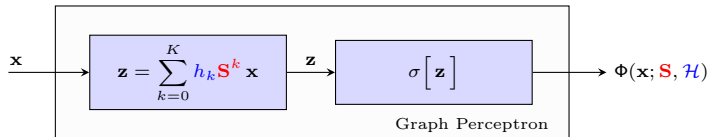
- ▶ Graph filters have **limited expressive power** because they can only learn linear maps
- ▶ An alternative choice for Φ is the set of **graph perceptrons** $\Rightarrow \Phi(\mathbf{x}) = \Phi(\mathbf{x}; \mathbf{S}, \mathbf{h}) = \sigma \left[\sum_{k=0}^K h_k \mathbf{S}^k \mathbf{x} \right]$



- ▶ Nonlinearity $\sigma(\cdot)$ is elementwise; e.g., $[\sigma(\mathbf{z})]_i = \max\{0, z_i\}$

Graph Perceptron

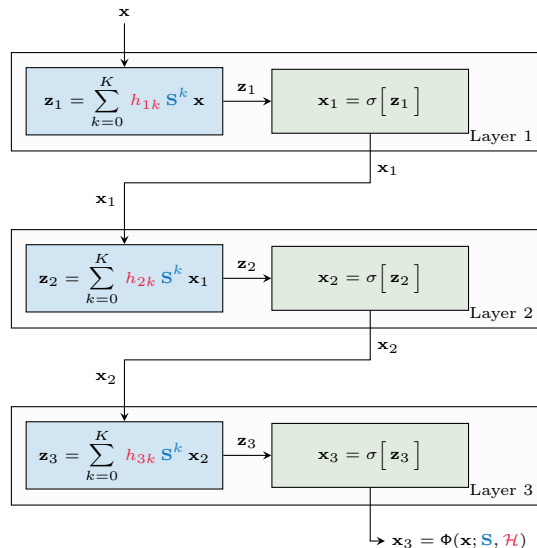
- ▶ Graph filters have **limited expressive power** because they can only learn linear maps
- ▶ An alternative choice for Φ is the set of **graph perceptrons** $\Rightarrow \Phi(\mathbf{x}) = \Phi(\mathbf{x}; \mathbf{S}, \mathbf{h}) = \sigma \left[\sum_{k=0}^K h_k \mathbf{S}^k \mathbf{x} \right]$



- ▶ Nonlinearity $\sigma(\cdot)$ is elementwise; e.g., $[\sigma(\mathbf{z})]_i = \max\{0, z_i\}$
- ▶ Optimal regressor restricted to perceptron class $\Rightarrow \mathbf{h}^* = \arg \min_{\mathbf{h}} \sum_{\mathbf{x} \in \mathcal{T}} \mathcal{L}(\Phi(\mathbf{x}; \mathbf{S}, \mathbf{h}))$
 $\Rightarrow$ Perceptron allows **learning of nonlinear maps** $\Rightarrow$ **More expressive**. Larger Representable Class

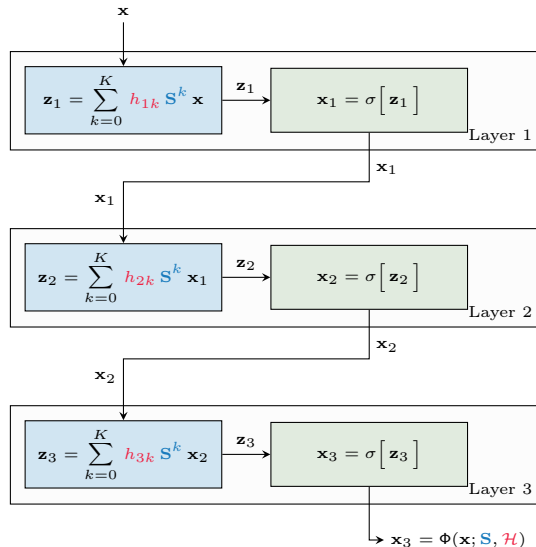
Graph Convolutional Neural Network (cont.)

- **Layer** a few **graph perceptrons** (3 in the figure)
 - ⇒ Feed the input signal $\mathbf{x}$ to Layer 1
 - ⇒ Connect output of Layer 1 to input of Layer 2
 - ⇒ And output of Layer 2 to input of Layer 3



Graph Convolutional Neural Network (cont.)

- **Layer** a few **graph perceptrons** (3 in the figure)
 - ⇒ Feed the input signal $\mathbf{x}$ to Layer 1
 - ⇒ Connect output of Layer 1 to input of Layer 2
 - ⇒ And output of Layer 2 to input of Layer 3
- Last layer output is the GNN output $\Rightarrow \Phi(\mathbf{x}; \mathbf{S}, \mathcal{H})$
 - ⇒ Parametrized on **filter tensor** $\mathcal{H} = [\mathbf{h}_1, \mathbf{h}_2, \mathbf{h}_3]$

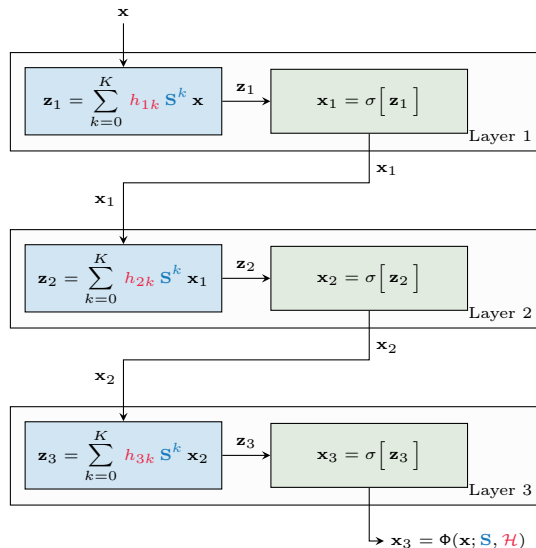


Graph Convolutional Neural Network (cont.)

- Layer a few **graph perceptrons** (3 in the figure)
 - ⇒ Feed the input signal $\mathbf{x}$ to Layer 1
 - ⇒ Connect output of Layer 1 to input of Layer 2
 - ⇒ And output of Layer 2 to input of Layer 3
- Last layer output is the GNN output $\Rightarrow \Phi(\mathbf{x}; \mathbf{S}, \mathcal{H})$
 - ⇒ Parametrized on **filter tensor** $\mathcal{H} = [\mathbf{h}_1, \mathbf{h}_2, \mathbf{h}_3]$
- Layer ℓ propagation rule

$$\mathbf{x}_\ell = \sigma[\mathbf{z}_\ell] = \sigma \left[\sum_{k=0}^K h_{\ell k} \mathbf{S}^k \mathbf{x}_{\ell-1} \right]$$

⇒ Trainable parameters: $\mathcal{H} = [\mathbf{h}_1, \dots, \mathbf{h}_L]$



Graph Convolutional Neural Network

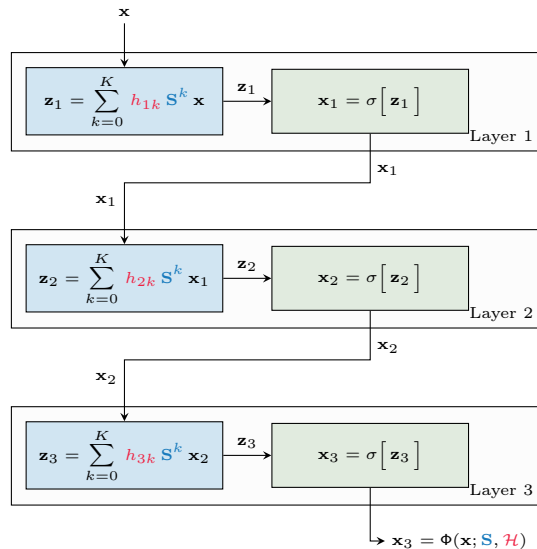
- Learn Optimal GNN tensor $\mathcal{H}^* = (\mathbf{h}_1^*, \mathbf{h}_2^*, \mathbf{h}_3^*)$ as

$$\mathcal{H}^* = \arg \min_{\mathcal{H}} \sum_{\mathbf{x} \in \mathcal{T}} \mathcal{L}(\Phi(\mathbf{x}; \mathbf{S}, \mathcal{H}))$$

- Optimization is over parameter tensor only. Graph $\mathbf{S}$ is given

$\Rightarrow$ Prior information given to the GNN

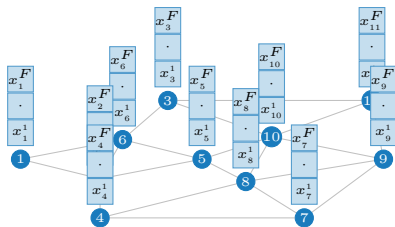
- Could also be over $\mathbf{S}$ but then the DoFs are too many



Accounting for Multiple Features

GCNN with Filter Banks

- Multiple feature graph signal $\Rightarrow$ Matrix $\mathbf{X}$ of size N (nodes) $\times$ F (features)
 - $\Rightarrow$ Row i collects all features at node $i \Rightarrow$ Local information at node i
 - $\Rightarrow$ Column $g = 1, \dots, F$ represents feature g at all nodes $\Rightarrow \mathbf{x}^g$ is a graph signal



$$\mathbf{X} = \begin{bmatrix} x_1^1 & x_1^2 & x_1^3 & \cdots & x_1^F \\ x_2^1 & x_2^2 & x_2^3 & \cdots & x_2^F \\ x_3^1 & x_3^2 & x_3^3 & \cdots & x_3^F \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ x_{10}^1 & x_{10}^2 & x_{10}^3 & \cdots & x_{10}^F \\ x_{11}^1 & x_{11}^2 & x_{11}^3 & \cdots & x_{11}^F \end{bmatrix}$$

fig. credit F. Gama

GCNN with Filter Banks (cont.)

- Convolution $\Rightarrow$ Linear operation, local information, distributed implementation

$$\mathbf{Z} = \sum_{k=0}^K \mathbf{S}^k \mathbf{X} \mathbf{H}_k$$

- Multiplication by $\mathbf{S}$ on the left $\Rightarrow$ Shifts each graph signal feature $\mathbf{S}\mathbf{x}^g$

$$\begin{bmatrix} 0 & s_{1,2} & 0 & \cdots & 0 \\ s_{2,1} & 0 & 0 & \cdots & 0 \\ 0 & 0 & 0 & \cdots & s_{3,11} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & s_{10,11} \\ 0 & 0 & s_{3,11} & \cdots & 0 \end{bmatrix} \begin{bmatrix} x_1^1 & x_1^2 & x_1^3 & \cdots & x_1^F \\ x_2^1 & x_2^2 & x_2^3 & \cdots & x_2^F \\ x_3^1 & x_3^2 & x_3^3 & \cdots & x_3^F \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ x_{10}^1 & x_{10}^2 & x_{10}^3 & \cdots & x_{10}^F \\ x_{11}^1 & x_{11}^2 & x_{11}^3 & \cdots & x_{11}^F \end{bmatrix}$$

GCNN with Filter Banks (cont.)

- Convolution $\Rightarrow$ Linear operation, local information, distributed implementation

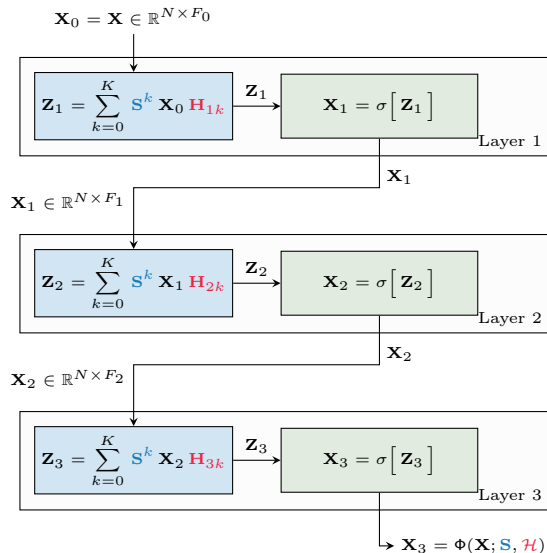
$$\mathbf{Z} = \sum_{k=0}^K \mathbf{S}^k \mathbf{X} \mathbf{H}_k$$

- Multiplication by $\mathbf{S}$ on the left $\Rightarrow$ Shifts each graph signal feature $\mathbf{S}\mathbf{x}^g$
- Multiplication by $\mathbf{H}$ on the right $\Rightarrow$ Linear combination of features at each node (No exchanges)

$$\begin{bmatrix} 0 & s_{1,2} & 0 & \cdots & 0 \\ s_{2,1} & 0 & 0 & \cdots & 0 \\ 0 & 0 & 0 & \cdots & s_{3,11} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & s_{10,11} \\ 0 & 0 & s_{3,11} & \cdots & 0 \end{bmatrix} \begin{bmatrix} x_1^1 & x_1^2 & x_1^3 & \cdots & x_1^F \\ x_2^1 & x_2^2 & x_2^3 & \cdots & x_2^F \\ x_3^1 & x_3^2 & x_3^3 & \cdots & x_3^F \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ x_{10}^1 & x_{10}^2 & x_{10}^3 & \cdots & x_{10}^F \\ x_{11}^1 & x_{11}^2 & x_{11}^3 & \cdots & x_{11}^F \end{bmatrix} \begin{bmatrix} h_k^{11} & h_k^{12} & \cdots & h_k^{1F_1} \\ h_k^{21} & h_k^{22} & \cdots & h_k^{2F_1} \\ h_k^{31} & h_k^{32} & \cdots & h_k^{3F_1} \\ \vdots & \vdots & \ddots & \vdots \\ h_k^{F1} & h_k^{F2} & \cdots & h_k^{FF_1} \end{bmatrix}$$

GCNN with Filter Banks (cont.)

- ▶ 3-layer GCNNs
- ▶ First layer is fed by $\mathbf{X}_0 = \mathbf{X}$
- ▶ Last layer output $\mathbf{Z} := \mathbf{X}_3 = \Phi(\mathbf{X}, \mathbf{S}, \mathcal{H})$
- ▶ Model parameters $\mathcal{H} = \{\mathbf{H}_{lk}\}_{lk}$



GCNN with Filter Banks (cont.)

- Multi feature graph neural networks $\Rightarrow \Phi(\mathbf{x}; \mathbf{S}, \mathcal{H}) = \mathbf{X}_L$

$$\mathbf{X}_\ell = \sigma \left[\sum_{k=0}^K \mathbf{S}^k \mathbf{X}_{\ell-1} \mathbf{H}_{\ell k} \right]$$

$\mathbf{X}_\ell$ graph signal at layer $\ell \Rightarrow$ Size $N \times F_\ell$

$\mathbf{H}_{\ell k}$ k th filter tap at layer $\ell \Rightarrow$ Size $F_{\ell-1} \times F_\ell \Rightarrow$ Learn $\sum_\ell K F_\ell F_{\ell-1}$ filter taps $\mathcal{H} = \{\mathbf{H}_{\ell k}\}$

$\mathbf{S}$ shift operator (given by the problem) $\Rightarrow$ Size $N \times N$

GCNN with Filter Banks (cont.)

- Multi feature graph neural networks $\Rightarrow \Phi(\mathbf{x}; \mathbf{S}, \mathcal{H}) = \mathbf{X}_L$

$$\mathbf{X}_\ell = \sigma \left[\sum_{k=0}^K \mathbf{S}^k \mathbf{X}_{\ell-1} \mathbf{H}_{\ell k} \right]$$

$\mathbf{X}_\ell$ graph signal at layer $\ell \Rightarrow$ Size $N \times F_\ell$

$\mathbf{H}_{\ell k}$ k th filter tap at layer $\ell \Rightarrow$ Size $F_{\ell-1} \times F_\ell \Rightarrow$ Learn $\sum_\ell K F_\ell F_{\ell-1}$ filter taps $\mathcal{H} = \{\mathbf{H}_{\ell k}\}$

$\mathbf{S}$ shift operator (given by the problem) $\Rightarrow$ Size $N \times N$

- Hyperparameters (design choices) $\Rightarrow$ Affect the number of learnable parameters
 - $\Rightarrow L$: number of layers
 - $\Rightarrow K$: number of filter taps on each layer
 - $\Rightarrow F_\ell$: number of features on each layer

Learning activity: Beyond a single graph filter

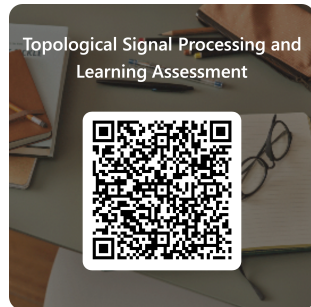
Q2.2: A company uses graph-based learning to classify products in a recommendation network. A graph filter captures local neighborhood information, but still struggles with the classification task. Which statement best explains why a deeper graph neural network may help?

► Instructions

- ⇒ Scan the QR code to open the activity
- ⇒ Work individually first
- ⇒ Then discuss your answer in pairs or groups of 3
- ⇒ Submit your final answer in Microsoft Forms

► Logistics

- ⇒ **Time:** 10 min
- ⇒ **Goal:** Explain why deeper graph models can outperform a single linear filter
- ⇒ **Product:** One submitted response per student + justification to your peers



<https://forms.office.com/e/yTag1Njapt>

Further reading

- ▶ (Gama et al) Graphs, Convolutions, and Neural Networks: From Graph Filters to Graph Neural Networks
⇒ Excluding Stability and Extensions
- ▶ (Isufi et al) Graph filters for processing and learning from network data
⇒ Ch. 3; Ch 6; Ch 7.1-7.4 (including)
- ▶ (Isufi et al) Graph filters for signal processing and machine learning on graphs network data
⇒ Sec. III; Sec. V-B; Sec. VIII (till VIII-A, including)
- ▶ (Ma and Tang) Deep Learning on Graphs https://web.njit.edu/~ym329/dlg_book/
⇒ Ch. 3 Foundations of Deep Learning (if you need a refresher)

Video material

- ▶ <https://www.youtube.com/@alelabalelab5337/courses>
⇒ Lectures 1-4

Backup Slides

Recursive Implementation of Graph Filters

- **Exploit** the recursive property $\mathbf{x}^{(k)} = \mathbf{S}^k \mathbf{x} = \mathbf{S}(\mathbf{S}^{k-1} \mathbf{x}) = \mathbf{S} \mathbf{x}^{(k-1)}$

Recursive Implementation of Graph Filters

► **Exploit** the recursive property $\mathbf{x}^{(k)} = \mathbf{S}^k \mathbf{x} = \mathbf{S}(\mathbf{S}^{k-1} \mathbf{x}) = \mathbf{S} \mathbf{x}^{(k-1)}$

⇒ Multiplication with $\mathbf{S}$ ⇒ neighbor shift/exchange ⇒ local

⇒ Exchange with neighbors former information $\mathbf{x}^{(k-1)}$

$$\mathbf{x}^{(k)} = \mathbf{S} \mathbf{x}^{(k-1)} = \mathbf{S}(\mathbf{S} \mathbf{x}^{(k-2)}) = \mathbf{S}(\mathbf{S}(\mathbf{S} \dots (\mathbf{S} \mathbf{x})))$$

Recursive Implementation of Graph Filters

- **Exploit** the recursive property $\mathbf{x}^{(k)} = \mathbf{S}^k \mathbf{x} = \mathbf{S}(\mathbf{S}^{k-1} \mathbf{x}) = \mathbf{S} \mathbf{x}^{(k-1)}$

⇒ Multiplication with $\mathbf{S}$ ⇒ neighbor shift/exchange ⇒ local

⇒ Exchange with neighbors former information $\mathbf{x}^{(k-1)}$

$$\mathbf{x}^{(k)} = \mathbf{S} \mathbf{x}^{(k-1)} = \mathbf{S}(\mathbf{S} \mathbf{x}^{(k-2)}) = \mathbf{S}(\mathbf{S}(\mathbf{S} \dots (\mathbf{S} \mathbf{x})))$$

- **Matrix-vector multiplication** $\mathbf{S} \mathbf{x}^{(k)}$

⇒ $\mathbf{S}$ is super sparse $\sim \mathcal{O}(|\mathcal{E}|)$

⇒ complexity of $\mathbf{S} \mathbf{x}$: $\mathcal{O}(|\mathcal{E}|)$

Recursive Implementation of Graph Filters

- **Exploit** the recursive property $\mathbf{x}^{(k)} = \mathbf{S}^k \mathbf{x} = \mathbf{S}(\mathbf{S}^{k-1} \mathbf{x}) = \mathbf{S} \mathbf{x}^{(k-1)}$
 - ⇒ Multiplication with $\mathbf{S}$ ⇒ neighbor shift/exchange ⇒ local
 - ⇒ Exchange with neighbors former information $\mathbf{x}^{(k-1)}$

$$\mathbf{x}^{(k)} = \mathbf{S} \mathbf{x}^{(k-1)} = \mathbf{S}(\mathbf{S} \mathbf{x}^{(k-2)}) = \mathbf{S}(\mathbf{S}(\mathbf{S} \dots (\mathbf{S} \mathbf{x})))$$

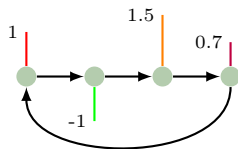
- **Matrix-vector multiplication** $\mathbf{S} \mathbf{x}^{(k)}$
 - ⇒ $\mathbf{S}$ is super sparse $\sim \mathcal{O}(|\mathcal{E}|)$
 - ⇒ complexity of $\mathbf{S} \mathbf{x} : \mathcal{O}(|\mathcal{E}|)$
 - ⇒ Complexity of filter output $\mathcal{O}(K|\mathcal{E}|)$
 - ⇒ Linear in #edges
 - ⇒ Distributable: local exchanges

Connection with CNNs

► Temporal signal shifting:

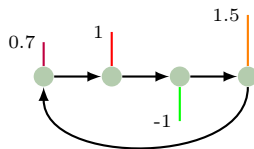
⇒ Shifting temporal signals = multiplying with shift operator $\mathbf{S}$

$$\mathbf{S} = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$



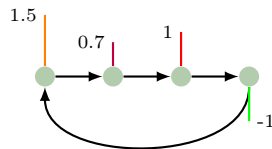
$$\mathbf{x} = [1, -1, 1.5, 0.7]^T$$

Original signal



$$\mathbf{x}^{(1)} = \mathbf{S}\mathbf{x} = [0.7, 1, -1, 1.5]^T$$

One-shift signal



$$\mathbf{x}^{(2)} = \mathbf{S}\mathbf{x}^{(1)} = [1.5, 0.7, 1, -1]^T$$

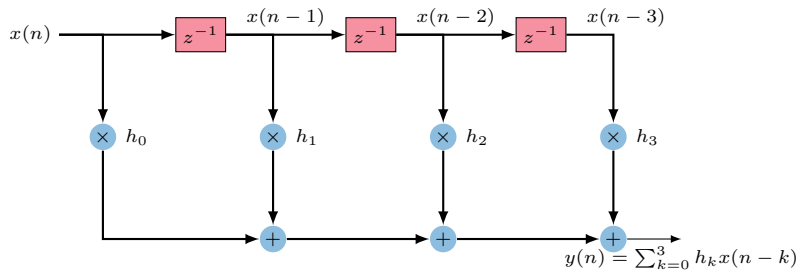
Two-shift signal

► Diffusion sequence $\mathbf{x}, \mathbf{S}\mathbf{x}, \mathbf{S}^2\mathbf{x}, \dots, \mathbf{S}^K\mathbf{x}$

Connection with CNNs

► Temporal Convolutional Filter

⇒ weighted shift and sum



► Third order convolutional filter

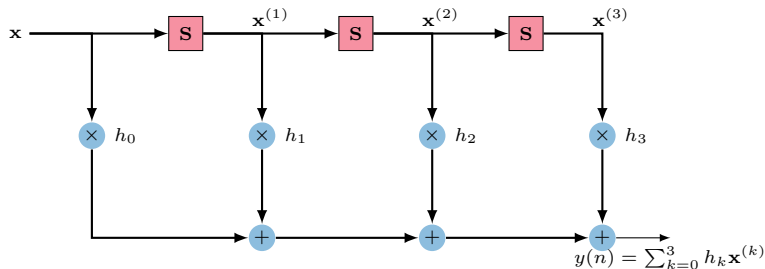
⇒ Coefficients: h_0, h_1, h_2, h_3

Connection with CNNs

- As graph convolutional filter

⇒ $\mathbf{x} = [x(1), x(2), x(3), x(4)]^\top$: time signals **stacked** as a **vector**

⇒ $\mathbf{y} = [y(1), y(2), y(3), y(4)]^\top$: **vector** of **output** time signals



- Similar to temporal CNNs, we stack graph convolutional filters with activation functions

GCNN with Filter Banks (cont.)

- Convolution $\Rightarrow$ Linear operation, local information, distributed implementation

$$\mathbf{Z} = \sum_{k=0}^K \mathbf{S}^k \mathbf{X} \mathbf{H}_k$$

- The convolution is now equivalent to the application of a bank of graph filters

GCNN with Filter Banks (cont.)

- Convolution $\Rightarrow$ Linear operation, local information, distributed implementation

$$\mathbf{Z} = \sum_{k=0}^K \mathbf{S}^k \mathbf{X} \mathbf{H}_k$$

- The convolution is now equivalent to the application of a bank of graph filters
- For each feature $\mathbf{x}^g$ we apply the (f, g) filter to obtain $\mathbf{z}^{fg} \Rightarrow$ Linear, local, distributed

$$\mathbf{z}^{fg} = \sum_{k=0}^K \mathbf{S}^k \mathbf{x}^g \mathbf{h}_k^{fg}$$

GCNN with Filter Banks (cont.)

- Convolution $\Rightarrow$ Linear operation, local information, distributed implementation

$$\mathbf{Z} = \sum_{k=0}^K \mathbf{S}^k \mathbf{X} \mathbf{H}_k$$

- For each feature $\mathbf{x}^g$ we apply the (f, g) filter to obtain $\mathbf{z}^{fg} \Rightarrow$ Linear, local, distributed

$$\mathbf{z}^{fg} = \sum_{k=0}^K \mathbf{S}^k \mathbf{x}^g h_k^{fg}$$

$\Rightarrow$ There are F_1 filters applied to each input $\mathbf{x}^g \Rightarrow$ There a total of $F \times F_1$ filters (size of $\mathbf{H}_k$)

GCNN with Filter Banks (cont.)

- Convolution $\Rightarrow$ Linear operation, local information, distributed implementation

$$\mathbf{Z} = \sum_{k=0}^K \mathbf{S}^k \mathbf{X} \mathbf{H}_k$$

- For each feature $\mathbf{x}^g$ we apply the (f, g) filter to obtain $\mathbf{z}^{fg} \Rightarrow$ Linear, local, distributed

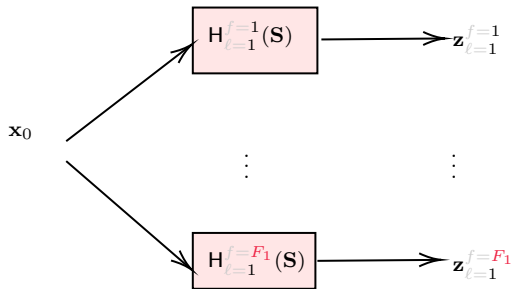
$$\mathbf{z}^{fg} = \sum_{k=0}^K \mathbf{S}^k \mathbf{x}^g h_k^{fg}$$

$\Rightarrow$ There are F_1 filters applied to each input $\mathbf{x}^g \Rightarrow$ There a total of $F \times F_1$ filters (size of $\mathbf{H}_k$)

- Aggregate the output of each g th filter across all F features $\Rightarrow \mathbf{z}^f = \sum_{g=1}^F \mathbf{z}^{fg} \Rightarrow \mathbf{Z} = [\mathbf{z}^1, \dots, \mathbf{z}^{F_1}]$

GCNN with Filter Banks (cont.)

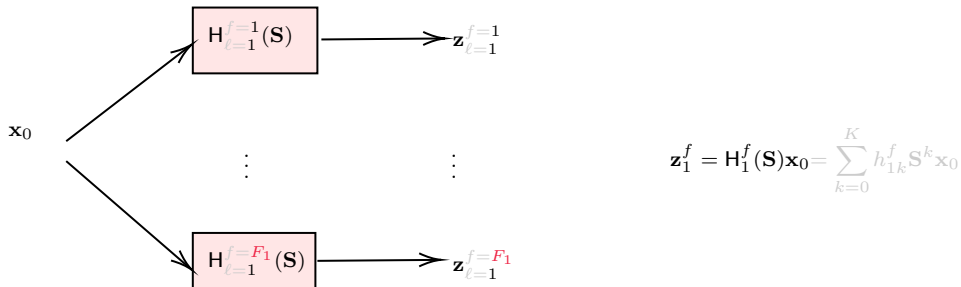
- Consider at the **first layer** $\ell = 1$ we have one feature signal as **input** $\mathbf{x}_0 := \mathbf{x}$ ($F = 1$)
- Process this signal with a **bank of F_1 filters**



$$\mathbf{z}_1^f = \mathbf{H}_1^f(\mathbf{S})\mathbf{x}_0 = \sum_{k=0}^K h_{1k}^f \mathbf{S}^k \mathbf{x}_0$$

GCNN with Filter Banks (cont.)

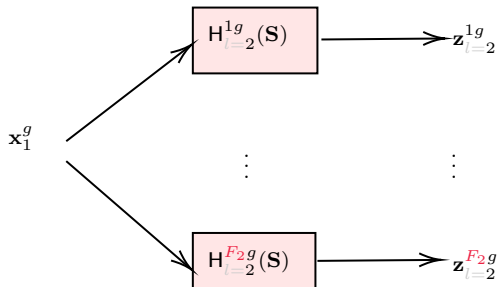
- Consider at the **first layer** $\ell = 1$ we have one feature signal as **input** $\mathbf{x}_0 := \mathbf{x}$ ($F = 1$)
- Process this signal with a **bank of F_1 filters**



- Since in the input we have only one feature $\Rightarrow$ there is **no** feature aggregation $\sum_g$
 $\Rightarrow$ Apply nonlinearity $\mathbf{x}_1^f := \sigma(\mathbf{z}_1^f)$ for $f = 1, \dots, F_1$

GCNN with Filter Banks (cont.)

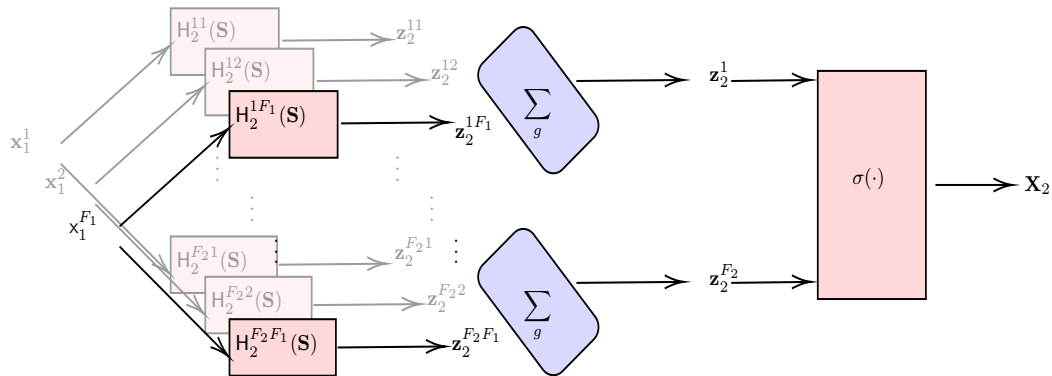
- At the **second layer** $\ell = 2$ we have the input $\mathbf{X}_1 = [\mathbf{x}_1^1, \dots, \mathbf{x}_1^g, \dots, \mathbf{x}_1^{F_1}] \in \mathbb{R}^{N \times F_1}$
- Process each column graph signal $\mathbf{x}_1^g$ with **F_2 filters**



$$\mathbf{z}_2^{fg} = \mathbf{H}_2^{fg}(\mathbf{S})\mathbf{x}_1^g = \sum_{k=0}^K h_{2k}^{fg} \mathbf{S}^k \mathbf{x}_1^g$$

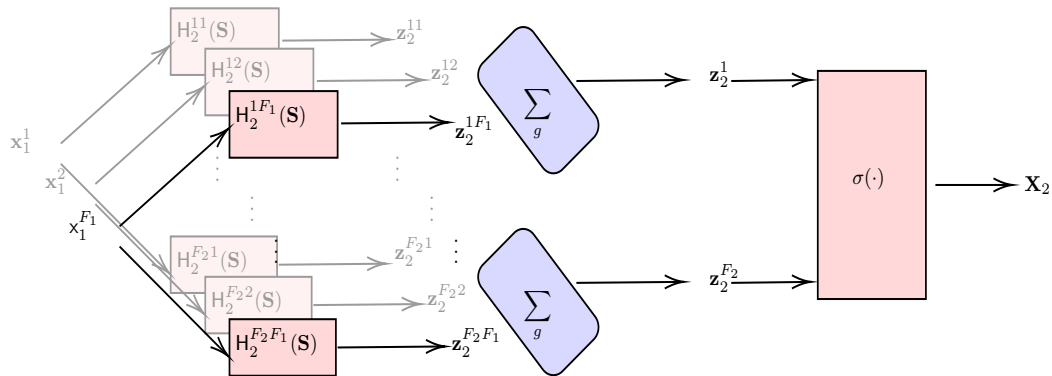
GCNN with Filter Banks

- Repeat these steps **for all** inputs $\mathbf{X}_1 = [\mathbf{x}_1^1, \dots, \mathbf{x}_1^g, \dots, \mathbf{x}_1^{F_1}] \in \mathbb{R}^{N \times F_1}$



GCNN with Filter Banks

- Repeat these steps **for all** inputs $\mathbf{X}_1 = [\mathbf{x}_1^1, \dots, \mathbf{x}_1^g, \dots, \mathbf{x}_1^{F_1}] \in \mathbb{R}^{N \times F_1}$



- Repeat this for all layers ℓ

GCNN with Filter Banks (cont.)



$$\mathbf{z}_\ell^f = \sum_{g=1}^{F_{\ell-1}} \mathbf{H}_\ell^{fg}(\mathbf{S}) \mathbf{x}_{\ell-1}^g = \sum_{g=1}^{F_{\ell-1}} \sum_{k=0}^K h_{\ell k}^{fg} \mathbf{S}^k \mathbf{x}_{\ell-1}^g$$

GCNN with Filter Banks (cont.)

►

$$\mathbf{z}_\ell^f = \sum_{g=1}^{F_{\ell-1}} \mathbf{H}_\ell^{fg}(\mathbf{S}) \mathbf{x}_{\ell-1}^g = \sum_{g=1}^{F_{\ell-1}} \sum_{k=0}^K h_{\ell k}^{fg} \mathbf{S}^k \mathbf{x}_{\ell-1}^g \xleftrightarrow{\text{Equivalent matrix form}} \mathbf{z}_\ell = \sum_{k=0}^K \mathbf{S}^k \mathbf{x}_{\ell-1} \mathbf{H}_{\ell k}$$

GCNN with Filter Banks (cont.)

►

$$\mathbf{z}_\ell^f = \sum_{g=1}^{F_{\ell-1}} \mathbf{H}_\ell^{fg}(\mathbf{S}) \mathbf{x}_{\ell-1}^g = \sum_{g=1}^{F_{\ell-1}} \sum_{k=0}^K h_{\ell k}^{fg} \mathbf{S}^k \mathbf{x}_{\ell-1}^g \xleftrightarrow{\text{Equivalent matrix form}} \mathbf{z}_\ell = \sum_{k=0}^K \mathbf{S}^k \mathbf{X}_{\ell-1} \mathbf{H}_{\ell k}$$

- All $F_{\ell-1} \times F_\ell$ filter parameters in $(\mathbf{H}_{\ell k})_{fg} = h_{\ell k}^{fg}$

GCNN with Filter Banks (cont.)

►

$$\mathbf{z}_\ell^f = \sum_{g=1}^{F_{\ell-1}} \mathbf{H}_\ell^{fg}(\mathbf{S}) \mathbf{x}_{\ell-1}^g = \sum_{g=1}^{F_{\ell-1}} \sum_{k=0}^K h_{\ell k}^{fg} \mathbf{S}^k \mathbf{x}_{\ell-1}^g \xleftrightarrow{\text{Equivalent matrix form}} \mathbf{z}_\ell = \sum_{k=0}^K \mathbf{S}^k \mathbf{x}_{\ell-1} \mathbf{H}_{\ell k}$$

- All $F_{\ell-1} \times F_\ell$ filter parameters in $(\mathbf{H}_{\ell k})_{fg} = h_{\ell k}^{fg}$
- The GCNN output is that of the last layer L

$$\mathbf{Z} := \mathbf{X}_L = \sigma[\mathbf{Z}_L] = \sigma \left[\sum_{k=0}^K \mathbf{S}^k \mathbf{X}_{L-1} \mathbf{H}_{Lk} \right]$$

GCNN with Filter Banks (cont.)



$$\mathbf{z}_\ell^f = \sum_{g=1}^{F_{\ell-1}} \mathbf{H}_\ell^{fg}(\mathbf{S}) \mathbf{x}_{\ell-1}^g = \sum_{g=1}^{F_{\ell-1}} \sum_{k=0}^K h_{\ell k}^{fg} \mathbf{S}^k \mathbf{x}_{\ell-1}^g \xleftrightarrow{\text{Equivalent matrix form}} \mathbf{z}_\ell = \sum_{k=0}^K \mathbf{S}^k \mathbf{x}_{\ell-1} \mathbf{H}_{\ell k}$$

► All $F_{\ell-1} \times F_\ell$ filter parameters in $(\mathbf{H}_{\ell k})_{fg} = h_{\ell k}^{fg}$

► The GCNN output is that of the last layer L

$$\mathbf{Z} := \mathbf{X}_L = \sigma[\mathbf{Z}_L] = \sigma \left[\sum_{k=0}^K \mathbf{S}^k \mathbf{X}_{L-1} \mathbf{H}_{Lk} \right]$$

$\Rightarrow$ Again, a function of the form $\mathbf{Z} := \Phi(\mathbf{X}, \mathbf{S}, \mathcal{H})$

$\Rightarrow$ Parameters $\mathcal{H} = \{\mathbf{h}_l^{fg}\}_{lfg}$

GCNN with Filter Banks (cont.)

Properties

- ▶ A **graph filter** has $K + 1$ parameters and complexity $\mathcal{O}(K|\mathcal{E}|)$

GCNN with Filter Banks (cont.)

Properties

- ▶ A **graph filter** has $K + 1$ parameters and complexity $\mathcal{O}(K|\mathcal{E}|)$
- ▶ A **GCNN layer** has F^2 graph filters – assuming $F_\ell = F \forall \ell$
 - $\Rightarrow F^2(K + 1)$ parameters
 - $\Rightarrow \mathcal{O}(F^2 K |\mathcal{E}|)$ complexity

GCNN with Filter Banks (cont.)

Properties

- ▶ A **graph filter** has $K + 1$ parameters and complexity $\mathcal{O}(K|\mathcal{E}|)$
- ▶ A **GCNN layer** has F^2 graph filters – assuming $F_\ell = F \forall \ell$
 - $\Rightarrow F^2(K + 1)$ parameters
 - $\Rightarrow \mathcal{O}(F^2 K |\mathcal{E}|)$ complexity
- ▶ A **GCNN** with L layers has
 - $\Rightarrow LF^2(K + 1)$ parameters
 - $\Rightarrow \mathcal{O}(LF^2 K |\mathcal{E}|)$ complexity

GCNN with Filter Banks (cont.)

Properties

- ▶ A **graph filter** has $K + 1$ parameters and complexity $\mathcal{O}(K|\mathcal{E}|)$
- ▶ A **GCNN layer** has F^2 graph filters – assuming $F_\ell = F \forall \ell$
 - $\Rightarrow F^2(K + 1)$ parameters
 - $\Rightarrow \mathcal{O}(F^2 K |\mathcal{E}|)$ complexity
- ▶ A **GCNN** with L layers has
 - $\Rightarrow LF^2(K + 1)$ parameters
 - $\Rightarrow \mathcal{O}(LF^2 K |\mathcal{E}|)$ complexity
- ▶ Parameters independent on nodes N
- ▶ Complexity linear in #edges $|\mathcal{E}|$

Why GCNNs work?

Why GCNN work?

A statistical learning perspective

- ▶ **Statistical learning**

- ⇒ Input variable $\mathbf{x}$ and output variable $\mathbf{y}$ are related by the joint distribution $p(\mathbf{x}, \mathbf{y})$

- ⇒ Relation exists naturally in the data

- ▶ **Goal:** predict output $\mathbf{y}$ from input $\mathbf{x}$ with the conditional distribution

$$\mathbf{x} \rightarrow p(\mathbf{y}|\mathbf{x}) \rightarrow \mathbf{y}$$

- **Statistical learning**

- ⇒ Input variable $\mathbf{x}$ and output variable $\mathbf{y}$ are related by the joint distribution $p(\mathbf{x}, \mathbf{y})$
- ⇒ Relation exists naturally in the data

- **Goal:** predict output $\mathbf{y}$ from input $\mathbf{x}$ with the conditional distribution

$$\mathbf{x} \rightarrow p(\mathbf{y}|\mathbf{x}) \rightarrow \mathbf{y}$$

- We do not have access to the true distribution $p(\mathbf{x}, \mathbf{y})$

- ⇒ **Find** a model $\Phi(\cdot)$ to estimate the output $\hat{\mathbf{y}}$ from the input $\mathbf{x}$

$$\mathbf{x} \rightarrow \Phi(\cdot) \rightarrow \hat{\mathbf{y}}$$

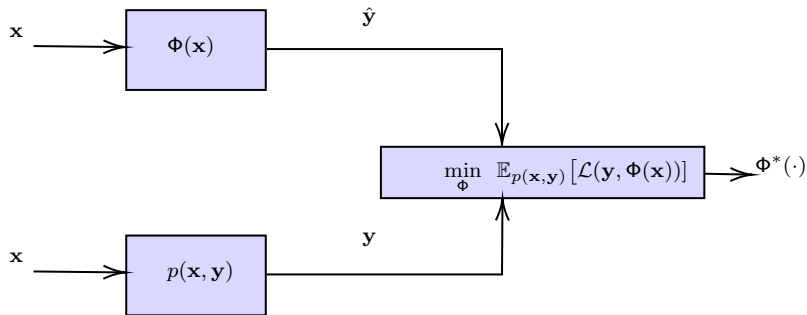
- **Loss function** $\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$ to measure the estimation cost

Why GCNN work? (cont.)

A statistical learning perspective

- **Expected loss** $\Rightarrow$ Minimize the expected loss over all inputs

$$\underset{\Phi}{\text{minimize}} \quad \mathbb{E}_{p(\mathbf{x}, \mathbf{y})} [\mathcal{L}(\mathbf{y}, \Phi(\mathbf{x}))]$$



Why GCNN work?(cont.)

An empirical learning perspective

► Average loss

⇒ We have no access to distribution $p(\mathbf{x}, \mathbf{y})$

⇒ We have access to finite data $\mathcal{T} = \{\mathbf{x}_r, \mathbf{y}_r\}_r$

⇒ Replace expectation by average mean (i.i.d. data assumption)

$$\underset{\Phi}{\text{minimize}} \quad \frac{1}{|\mathcal{T}|} \sum_r \mathcal{L}(\mathbf{y}_r, \Phi(\mathbf{x}_r))$$

Why GCNN work?(cont.)

An empirical learning perspective

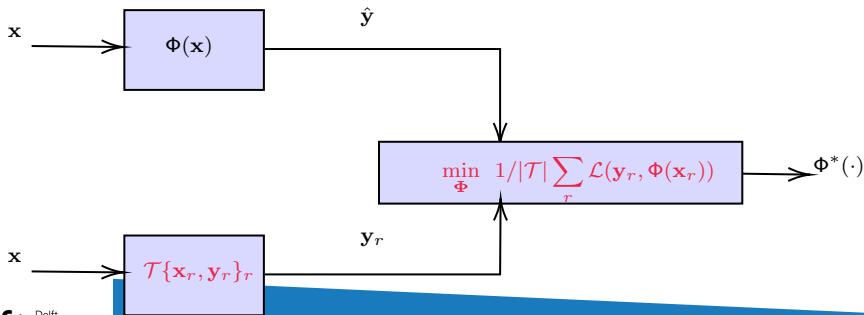
► Average loss

⇒ We have no access to distribution $p(\mathbf{x}, \mathbf{y})$

⇒ We have access to finite data $\mathcal{T} = \{\mathbf{x}_r, \mathbf{y}_r\}_r$

⇒ Replace expectation by average mean (i.i.d. data assumption)

$$\underset{\Phi}{\text{minimize}} \quad \frac{1}{|\mathcal{T}|} \sum_r \mathcal{L}(\mathbf{y}_r, \Phi(\mathbf{x}_r))$$



Why GCNN work?(cont.)

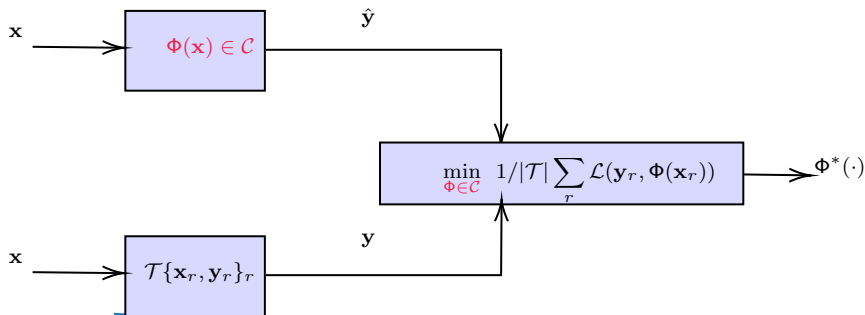
An empirical learning perspective

► Class function

⇒ We cannot solve for arbitrary function $\Phi(\cdot)$

⇒ Constraint the family of functions to a class $\Phi(\cdot) \in \mathcal{C}$

$$\underset{\Phi \in \mathcal{C}}{\text{minimize}} \quad \frac{1}{|\mathcal{T}|} \sum_r \mathcal{L}(\mathbf{y}_r, \Phi(\mathbf{x}_r))$$



Why GCNN work? (cont.)

An empirical learning perspective

► Components of Empirical Risk Minimization

⇒ Data $\mathcal{T} = \{\mathbf{x}_r, \mathbf{y}_r\}$

⇒ Family model $\Phi(\cdot) \in \mathcal{C}$

⇒ Loss function $\mathcal{L}(\cdot)$

⇒ Depending on what we know about the data we choose the family model (inductive bias)

Why GCNN work? (cont.)

An empirical learning perspective

► Components of Empirical Risk Minimization

⇒ Data $\mathcal{T} = \{\mathbf{x}_r, \mathbf{y}_r\}$

⇒ Family model $\Phi(\cdot) \in \mathcal{C}$

⇒ Loss function $\mathcal{L}(\cdot)$

⇒ Depending on what we know about the data we choose the family model (inductive bias)

► Class function ⇒ Family of functions: parametric $\Phi(\mathcal{H}, \mathbf{x}) \in \mathcal{C}$

⇒ Finding optimal function $\Phi^*(\cdot)$ implies finding optimal parameters $\mathcal{H}^*$

$$\underset{\mathcal{H}}{\text{minimize}} \quad \frac{1}{|\mathcal{T}|} \sum_r \mathcal{L}(\mathbf{y}_r, \Phi(\mathcal{H}, \mathbf{x}_r))$$

Why GCNN work? (cont.)

An empirical learning perspective

► Components of Empirical Risk Minimization

⇒ Data $\mathcal{T} = \{\mathbf{x}_r, \mathbf{y}_r\}$

⇒ Family model $\Phi(\cdot) \in \mathcal{C}$

⇒ Loss function $\mathcal{L}(\cdot)$

⇒ Depending on what we know about the data we choose the family model (inductive bias)

► Class function ⇒ Family of functions: parametric $\Phi(\mathcal{H}, \mathbf{x}) \in \mathcal{C}$

⇒ Finding optimal function $\Phi^*(\cdot)$ implies finding optimal parameters $\mathcal{H}^*$

$$\underset{\mathcal{H}}{\text{minimize}} \quad \frac{1}{|\mathcal{T}|} \sum_r \mathcal{L}(\mathbf{y}_r, \Phi(\mathcal{H}, \mathbf{x}_r))$$

► Goal 1: good fit $\Phi(\mathcal{H}^*, \mathbf{x})$ in the training set $\mathcal{T}$

Goal 2: good generalization $\Phi(\mathcal{H}^*, \mathbf{x})$ for unseen data $\mathbf{x} \notin \mathcal{T}$

► We know the data are related by a graph ⇒ that is our inductive bias

Why GCNN work? (cont.)

An inductive bias perspective

- ▶ Let's ignore the graph and process the data with multi-layer perceptrons
- ▶ Express function $\Phi(\mathcal{H}, \mathbf{x})$ as a cascade of layered functions

$$\Phi(\mathcal{H}, \mathbf{x}) = \Phi_3(\mathcal{H}_3, \Phi_2(\mathcal{H}_2, \Phi_1(\mathcal{H}_1, \mathbf{x})))$$

$\Rightarrow$ Parameters $\mathcal{H} = \{\mathcal{H}_1, \dots, \mathcal{H}_L\}$ in all layers

Why GCNN work? (cont.)

An inductive bias perspective

- ▶ Let's ignore the graph and process the data with multi-layer perceptrons
- ▶ Express function $\Phi(\mathcal{H}, \mathbf{x})$ as a cascade of layered functions

$$\Phi(\mathcal{H}, \mathbf{x}) = \Phi_3(\mathcal{H}_3, \Phi_2(\mathcal{H}_2, \Phi_1(\mathcal{H}_1, \mathbf{x})))$$

⇒ Parameters $\mathcal{H} = \{\mathcal{H}_1, \dots, \mathcal{H}_L\}$ in all layers

- ▶ Class function

⇒ Make each function $\Phi_\ell(\mathcal{H}_\ell, \cdot)$ a perceptron

⇒ $\hat{\mathbf{y}} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$

⇒ Parameters $\mathcal{H}_\ell = \{\mathbf{W}_\ell, \mathbf{b}_\ell\}$

⇒ Pointwise nonlinearity $\sigma(\cdot)$

Why GCNN work? (cont.)

An inductive bias perspective

- ▶ Let's ignore the graph and process the data with multi-layer perceptrons
- ▶ Express function $\Phi(\mathcal{H}, \mathbf{x})$ as a cascade of layered functions

$$\Phi(\mathcal{H}, \mathbf{x}) = \Phi_3(\mathcal{H}_3, \Phi_2(\mathcal{H}_2, \Phi_1(\mathcal{H}_1, \mathbf{x})))$$

⇒ Parameters $\mathcal{H} = \{\mathcal{H}_1, \dots, \mathcal{H}_L\}$ in all layers

- ▶ **Class function**

⇒ Make each function $\Phi_\ell(\mathcal{H}_\ell, \cdot)$ a perceptron

⇒ $\hat{\mathbf{y}} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$

⇒ Parameters $\mathcal{H}_\ell = \{\mathbf{W}_\ell, \mathbf{b}_\ell\}$

⇒ Pointwise nonlinearity $\sigma(\cdot)$

- ▶ **Multi-layer perceptron:** cascade perceptrons ⇒ improves expressivity

⇒ $\mathbf{x}_\ell = \sigma(\mathbf{W}_\ell \mathbf{x}_{\ell-1} + \mathbf{b}_\ell)$

Why GCNN work? (cont.)

An inductive bias perspective

- ▶ Multi-layer perceptron $\Rightarrow \mathbf{x}_l = \sigma(\mathbf{W}_l \mathbf{x}_{l-1} + \mathbf{b}_l)$
 - $\Rightarrow$ Unrolling recursion

$$\begin{aligned}\mathbf{x}_L &= \sigma(\mathbf{W}_L \mathbf{x}_{L-1} + \mathbf{b}_L) \\ &= \sigma(\mathbf{W}_L \sigma(\mathbf{W}_{L-1} \mathbf{x}_{L-2} + \mathbf{b}_{L-1}) + \mathbf{b}_L) \\ &= \sigma(\mathbf{W}_L \sigma(\mathbf{W}_{L-1} \sigma(\dots \sigma(\mathbf{W}_1 \mathbf{x}_0 + \mathbf{b}_0) + \mathbf{b}_{L-1}) + \mathbf{b}_L))\end{aligned}$$

Why GCNN work? (cont.)

An inductive bias perspective

- Multi-layer perceptron $\Rightarrow \mathbf{x}_l = \sigma(\mathbf{W}_l \mathbf{x}_{l-1} + \mathbf{b}_l)$
 - $\Rightarrow$ Unrolling recursion

$$\begin{aligned}\mathbf{x}_L &= \sigma(\mathbf{W}_L \mathbf{x}_{L-1} + \mathbf{b}_L) \\ &= \sigma(\mathbf{W}_L \sigma(\mathbf{W}_{L-1} \mathbf{x}_{L-2} + \mathbf{b}_{L-1}) + \mathbf{b}_L) \\ &= \sigma(\mathbf{W}_L \sigma(\mathbf{W}_{L-1} \sigma(\dots \sigma(\mathbf{W}_1 \mathbf{x}_0 + \mathbf{b}_0) + \mathbf{b}_{L-1}) + \mathbf{b}_L)\end{aligned}$$

- $\Rightarrow \mathbf{x}_L$ depends on $\mathbf{x}_0$ through a composition of linear functions and pointwise nonlinearities
- $\Rightarrow$ MLP fails in high dimensional data
- $\Rightarrow$ if layers have dimensions $\sim \mathcal{O}(N)$ $\Rightarrow \dim(\mathbf{W}_l) \sim \mathcal{O}(N^2)$, e.g., $N = 1000 \rightarrow \mathcal{O}(10^6)$
- $\Rightarrow$ complexity $\mathcal{O}(N^2)$

Why GCNN work? (cont.)

An inductive bias perspective

- ▶ Multi-layer perceptron $\Rightarrow \mathbf{x}_l = \sigma(\mathbf{W}_l \mathbf{x}_{l-1} + \mathbf{b}_l)$
 - $\Rightarrow$ Unrolling recursion

$$\begin{aligned}\mathbf{x}_L &= \sigma(\mathbf{W}_L \mathbf{x}_{L-1} + \mathbf{b}_L) \\ &= \sigma(\mathbf{W}_L \sigma(\mathbf{W}_{L-1} \mathbf{x}_{L-2} + \mathbf{b}_{L-1}) + \mathbf{b}_L) \\ &= \sigma(\mathbf{W}_L \sigma(\mathbf{W}_{L-1} \sigma(\dots \sigma(\mathbf{W}_1 \mathbf{x}_0 + \mathbf{b}_0) + \mathbf{b}_{L-1}) + \mathbf{b}_L)\end{aligned}$$

- $\Rightarrow \mathbf{x}_L$ depends on $\mathbf{x}_0$ through a composition of linear functions and pointwise nonlinearities
- $\Rightarrow$ MLP fails in high dimensional data
- $\Rightarrow$ if layers have dimensions $\sim \mathcal{O}(N)$ $\Rightarrow \dim(\mathbf{W}_l) \sim \mathcal{O}(N^2)$, e.g., $N = 1000 \rightarrow \mathcal{O}(10^6)$
- $\Rightarrow$ complexity $\mathcal{O}(N^2)$
- ▶ Need to exploit structure in data $\Rightarrow$ structure acts as an inductive bias
 - $\Rightarrow$ reduce complexity and parameters with GCNNs
 - $\Rightarrow$ inductive transference $\Rightarrow$ new node comes - no need to retrain

Why GCNN work? (cont.)

An inductive bias perspective

- ▶ Graph as inductive bias
 - ⇒ Leverage that data are **related** through the graph
 - ⇒ **Sparse** and local connectivity between nodes
 - ⇒ Coupling between data and topology

Why GCNN work? (cont.)

An inductive bias perspective

- ▶ Graph as inductive bias
 - ⇒ Leverage that data are **related** through the graph
 - ⇒ **Sparse** and local connectivity between nodes
 - ⇒ Coupling between data and topology
- ▶ Graph convolutional filters capture structure in data

$$\mathbf{z} = \sum_{k=0}^K h_k \mathbf{S}^k \mathbf{x}$$

- ⇒ Linear models
- ⇒ Family of functions $\Phi(\mathbf{x}, \mathbf{S}) \in \mathcal{C}$ to $\Phi(\mathbf{x}, \mathbf{S}, \mathbf{h}) = \mathbf{H}(\mathbf{S})\mathbf{x} = \sum_{k=0}^K h_k \mathbf{S}^k \mathbf{x}$
- ⇒ Parameters $\mathbf{h} = h_0, \dots, h_K$ independent of $\dim(\mathbf{x})$ $\Rightarrow$ shared among nodes
- ⇒ Complexity: Graph sparsity $\mathcal{O}(K|\mathcal{E}|)$
- ▶ GCNNs improve the expressive power of graph convolutional filters

Why GCNN work? (cont.)

An inductive bias perspective

- Since the GNN is a particular case of a fully connected NN, the latter attains a smaller cost

$$\min_{\mathcal{H}} \sum_{(\mathbf{x}, \mathbf{y}) \in \mathcal{T}} \mathcal{L}(\Phi(\mathbf{x}; \mathcal{H}), \mathbf{y}) \leq \min_{\mathcal{H}} \sum_{(\mathbf{x}, \mathbf{y}) \in \mathcal{T}} \mathcal{L}(\Phi(\mathbf{x}; \mathbf{S}, \mathcal{H}), \mathbf{y})$$

Why GCNN work? (cont.)

An inductive bias perspective

- ▶ Since the GNN is a particular case of a fully connected NN, the latter attains a smaller cost

$$\min_{\mathcal{H}} \sum_{(\mathbf{x}, \mathbf{y}) \in \mathcal{T}} \mathcal{L}(\Phi(\mathbf{x}; \mathcal{H}), \mathbf{y}) \leq \min_{\mathcal{H}} \sum_{(\mathbf{x}, \mathbf{y}) \in \mathcal{T}} \mathcal{L}(\Phi(\mathbf{x}; \mathbf{S}, \mathcal{H}), \mathbf{y})$$

- ▶ The fully connected NN does better. But this holds for the training set
- ▶ In practice, the GNN does better because it generalizes better to unseen signals / graphs
 - ⇒ Because it exploits internal symmetries of graph signals codified in the graph shift operator